

On this page



Manage inventory quantities and states

[📄 Install AI Toolkit](#) | [✈️ Ask about this page](#) | [📄 Copy MD](#)

This guide shows you how to use the GraphQL Admin API to sync and maintain accurate inventory quantities, and transition products to different inventory states.

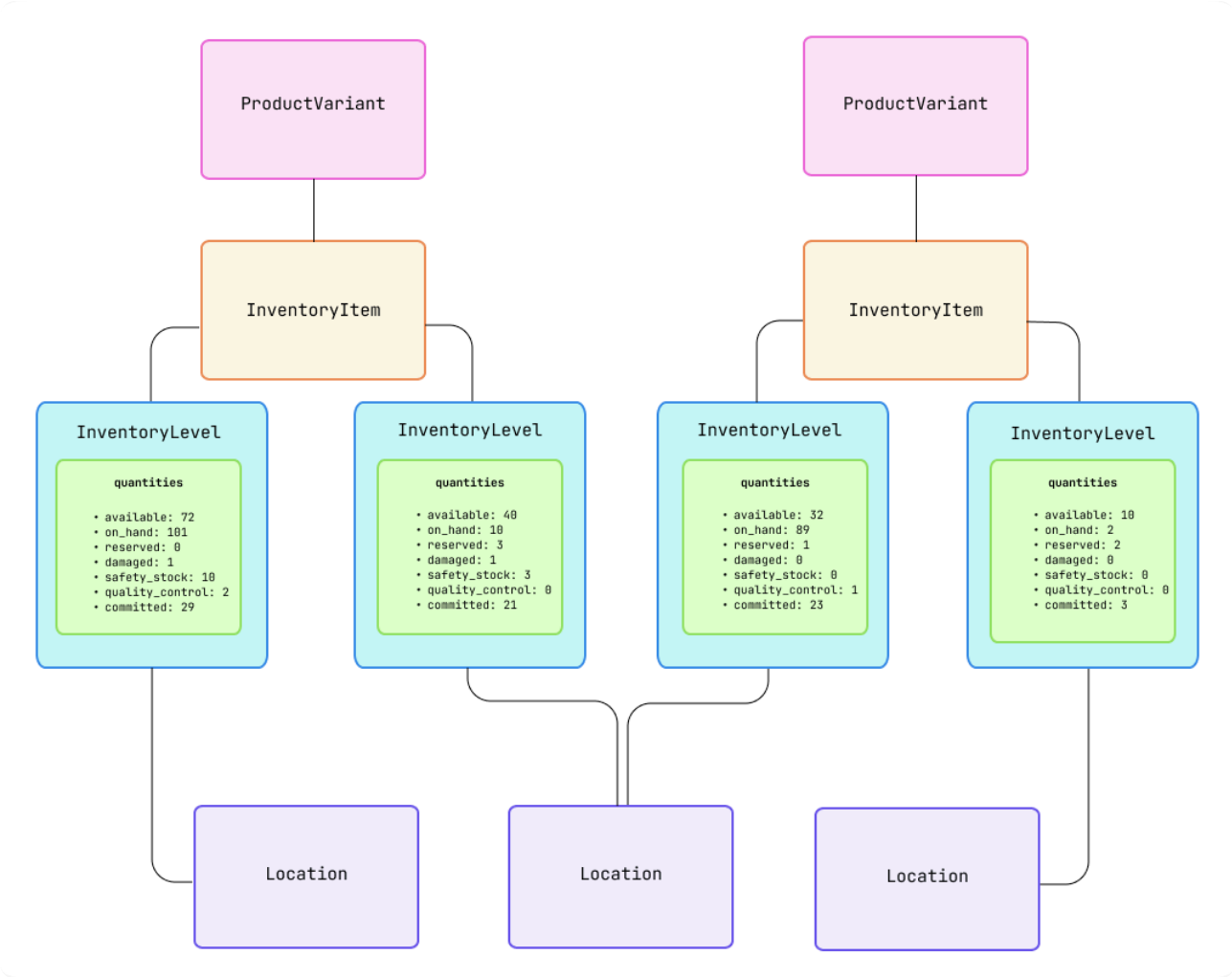
Requirements

- Your app can make [authenticated requests](#) to the GraphQL Admin API.
- Your app has the `read_inventory` and `write_inventory` [access scopes](#). Learn how to [configure your access scopes using Shopify CLI](#).
- Your store has existing [products](#).

[docs](#)[Apps](#)[Storefronts](#)[Agents](#)[Refer](#)[Ask assistant](#)[Help](#)[Log in](#)[APPS](#)[Build](#)[Design](#)[Launch](#)

Inventory object relationships

Before you begin managing inventory quantities and states, it's helpful to understand the relationships between inventory objects. The following diagram illustrates the relationships between inventory objects in the GraphQL Admin API:



API object	Description
ProductVariant	Contains merchandising information, such as price, description, and images. Think of this as the product information that you might want a customer to see. All product variants have a 1:1 relationship with their associated inventory item.
InventoryItem	Contains information about the physical product, such as its SKU. Think of this as the backend information used for managing inventory, shipping, and fulfillments. Inventory items are associated with one or many inventory levels. An inventory item has an inventory level for each location where the item is stocked.
InventoryLevel	The actual quantities of an item that are in a specific inventory state . Inventory levels connect one inventory item to one location. Each inventory level holds the available quantity for its inventory item at the associated location.
Location	A geographical location where a merchant does business, such as a retail store or warehouse.

API object	Description
	Locations can have many inventory levels. Each location has one inventory level for each inventory item that the location stocks.

Step 1: Retrieve IDs for inventory locations and inventory items

You can use the [locations](#) query to retrieve the IDs of inventory locations and inventory items. To retrieve inventory at fulfillment service app locations, you need to use the [shop](#) query instead of the `locations` query.

An inventory level represents the quantities of an inventory item that's associated with a location. An inventory item represents the goods that are available to be shipped to a customer. You'll use the IDs for inventory levels and inventory items to query inventory quantities in [the next step](#).

`locations` query

The following example shows how to retrieve the first three locations and their associated inventory levels. The response returns a set of parameterized global IDs.

Note

The `InventoryLevel` object has a global ID that contains parameters because the `InventoryLevel` object is associated with the `InventoryItem` object. For example, if the `InventoryLevel` object's ID is 69407473686 and the `InventoryItem` object's ID is 32889739542550, then the global ID resolves to the following structure:

```
gid://shopify/InventoryLevel/69407473686?  
inventory_item_id=32889739542550.
```

```
POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json
```

GraphQL query



```
1 query {  
2   # Retrieves the first three locations on a shop.  
3   locations(first: 3) {  
4     edges {  
5       node {
```

```

6      # The ID of the location.
7      id
8      # Retrieves the first three inventory levels associated with
9      inventoryLevels(first: 3) {
10         edges {
11             node {
12                 id
13             }
14         }
15     }
16 }
17 }
18 }
19 }

```

JSON response



```

1  {
2    "data": {
3      "locations": {
4        "edges": [
5          {
6            "node": {
7              "id": "gid://shopify/Location/35239591958",
8              "inventoryLevels": {
9                "edges": [
10                 {
11                   "node": {
12                     "id": "gid://shopify/InventoryLevel/69407473686?i
13                   }
14                 },
15                 {
16                   "node": {
17                     "id": "gid://shopify/InventoryLevel/69407473686?i
18                   }
19                 },
20                 {
21                   "node": {
22                     "id": "gid://shopify/InventoryLevel/69407473686?i
23                   }
24                 }
25               ]
26             }
27           }
28         },
29         {
30           "node": {
31             "id": "gid://shopify/Location/52187463702",
32             "inventoryLevels": {

```

```

33         "edges": [
34             {
35                 "node": {
36                     "id": "gid://shopify/InventoryLevel/86381920278?i
37                 }
38             },
39             {
40                 "node": {
41                     "id": "gid://shopify/InventoryLevel/86381920278?i
42                 }
43             }
44         ]
45     },
46 }
47 },
48 {
49     "node": {
50         "id": "gid://shopify/Location/35259777046",
51         "inventoryLevels": {
52             "edges": [
53                 {
54                     "node": {
55                         "id": "gid://shopify/InventoryLevel/69427953686?i
56                     }
57                 },
58                 {
59                     "node": {
60                         "id": "gid://shopify/InventoryLevel/69427953686?i
61                     }
62                 },
63                 {
64                     "node": {
65                         "id": "gid://shopify/InventoryLevel/69427953686?i
66                     }
67                 }
68             ]
69         }
70     }
71 }
72 ]
73 }
74 }
75 }

```

shop query

The following example shows how to retrieve the first three fulfillment service app locations and their associated inventory levels. The response returns a set of parameterized global IDs.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL query



```

1 query {
2   shop {
3     # Retrieves inventory at fulfillment service app locations
4     fulfillmentServices {
5       location {
6         # The ID of the location.
7         id
8         # Retrieves the first three inventory levels associated with each location.
9         inventoryLevels(first: 3) {
10          edges {
11            node {
12              id
13            }
14          }
15        }
16      }
17    }
18  }
19 }

```

JSON response



```

1 {
2   "data": {
3     "shop": {
4       "fulfillmentServices": [
5         {
6           "location": {
7             "id": "gid://shopify/Location/35239591958",
8             "inventoryLevels": {
9               "edges": [
10                {
11                  "node": {
12                    "id": "gid://shopify/InventoryLevel/69407473686?i=1"
13                  }
14                },
15                {
16                  "node": {
17                    "id": "gid://shopify/InventoryLevel/69407473686?i=2"
18                  }
19                }
20              ]
21            }
22          }
23        }
24      ]
25    }
26  }
27 }

```

```

19         },
20         {
21             "node": {
22                 "id": "gid://shopify/InventoryLevel/69407473686?i
23             }
24         }
25     ]
26 }
27 }
28 },
29 {
30     "location": {
31         "id": "gid://shopify/Location/67861446678",
32         "inventoryLevels": {
33             "edges": []
34         }
35     }
36 }
37 ]
38 }
39 },
40 ,

```

Step 2: Query inventory quantities

You can use the following queries to retrieve inventory quantities:

- [Query inventory quantities for an item at a specific location](#)
- [Query inventory quantities for an item at multiple locations](#)

Query inventory quantities for an item at a specific location

You can use the [inventoryLevel](#) query to retrieve inventory quantities for an item at a specific location.

Each inventory level belongs to one inventory item and has one location. For every location where an inventory item can be stocked, there's an inventory level that represents the inventory item's quantities relating to that location.

The following example query retrieves the quantity of an inventory item that's available at a specific location. The response shows 11 items that are available at the location.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL query



```
1 query {
2   inventoryLevel(id:"gid://shopify/InventoryLevel/69407473686?inventory
3     id
4     # The quantities field takes an array of inventory states, which in
5     quantities(names: ["available"]) {
6       name
7       quantity
8     }
9   item {
10     id
11   }
12   location {
13     id
14   }
15   createdAt
16   updatedAt
17   canDeactivate
18 }
19 }
```

JSON response



```
1 {
2   "data": {
3     "inventoryLevel": {
4       "id": "gid://shopify/InventoryLevel/69407473686?inventory_item_
5       "quantities": [
6         {
7           "name": "available",
8           "quantity": 11
9         }
10      ],
11     "item": {
12       "id": "gid://shopify/InventoryItem/32889739542550"
13     },
14     "location": {
15       "id": "gid://shopify/Location/35239591958"
16     },
17     "createdAt": "2020-05-11T23:37:38Z",
18     "updatedAt": "2020-08-28T05:57:20Z",
```

```

19         "canDeactivate": false
20     }
21 }
22 ,

```

Query inventory quantities for an item at multiple locations

You can use the [inventoryItem](#) query to retrieve inventory quantities for an item at multiple locations.

The following example query retrieves the quantity of an inventory item that are available, on-hand, reserved, and committed at the first five locations. The response shows 72 available items, 101 on-hand items, 0 reserved items, and 29 committed items at the locations.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL query



```

1  {
2    inventoryItem(id: "gid://shopify/InventoryItem/32889739542550") {
3      # Retrieves the first five inventory levels.
4      inventoryLevels(first: 5) {
5        edges {
6          node {
7            # The quantities field takes an array of inventory states, with
8            quantities(names: ["available", "on_hand", "reserved", "commi
9              name
10             quantity
11           }
12         }
13       }
14     }
15   }
16 }

```

JSON response



```

1  {
2    "data": {
3      "inventoryItem": {
4        "inventoryLevels": {
5          "edges": [
6            {
7              "node": {
8                "quantities": [
9                  {

```

```

10         "name": "available",
11         "quantity": 72
12     },
13     {
14         "name": "on_hand",
15         "quantity": 101
16     },
17     {
18         "name": "reserved",
19         "quantity": 0
20     },
21     {
22         "name": "committed",
23         "quantity": 29
24     }
25 ]
26 }
27 }
28 ]
29 }
30 }
31 }
32 }
```

Step 3: Manage inventory quantities

You can use the following mutations to manage inventory quantities:

- [Activate inventory at a location](#): Activate an inventory item at a location to make it available for sale or fulfillment there.
- [Set inventory quantities on hand](#): Explicitly set the quantity of inventory that's in the `available` or `on-hand` state.
- [Adjust inventory quantities](#): Add or remove quantities of inventory that are in the `available` state.
- [Move inventory quantities between states](#): Transition inventory quantities between the `available` state and the `reserved`, `damaged`, `safety_stock`, and `quality_control` states, or between unavailable states (`reserved`, `damaged`, `safety_stock`, and `quality_control`).

- [Schedule changes to inventory quantities](#): Create a scheduled change that states the dates associated with upcoming changes to inventory quantities.

Before API version `2026-04`, `inventorySetQuantities`, `inventoryAdjustQuantities`, and `inventoryMoveQuantities` returned an `ITEM_NOT_STOCKED_AT_LOCATION` error if the inventory item wasn't stocked at the location.

In API version `2026-04` and later, quantity mutations can create an inactive `InventoryLevel` and store the quantity. The item isn't available for sale or fulfillment from that location until it's activated there. Your app should explicitly activate the inventory item at the location with [inventoryActivate](#) or [inventoryBulkToggleActivation](#). If an inactive inventory level already has quantities, then activating it without `available` or `onHand` preserves those quantities.

Note

You can't use the GraphQL Admin API to adjust or move inventory quantities in the `committed` state. Inventory quantities in the `committed` state are managed by Shopify through actions such as creating and fulfilling orders, reserving inventory on draft orders, and marking transfers or shipments as ready to ship.

Activate inventory at a location

Use [inventoryActivate](#) to activate one inventory item at one location. Use [inventoryBulkToggleActivation](#) to activate one inventory item across multiple locations.

While you update your app, users with inventory permissions can use the Shopify admin as a temporary fallback:

1. From the Shopify admin, go to **Products**, and then open the product.
2. In the **Variants** section, open the variant.
3. In the **Inventory** section, click **Edit locations**.
4. Check the locations where the variant is stocked.
5. Click **Save**.
6. Review the **Available** and **On hand** quantities for the selected locations.

Set inventory quantities on hand

The `on_hand` state represents the total number of units that are physically stocked at a location. The `on_hand` state equals the sum of inventory quantities in the following states:

- `available`
 - `committed`
 - `reserved`
- `damaged`
 - `safety_stock`
 - `quality_control`

You set inventory in the `on_hand` state to maintain an accurate count of physical inventory. For example, you might need to set inventory in the `on_hand` state in the following scenarios:

- Inventory was received outside of a purchase order
- A stocktake was completed and inventory needs to be adjusted accordingly

You can use the [inventorySetQuantities](#) mutation to set available and on-hand inventory quantities at a location to a given value. In the `reason` field in the mutation's input, you can specify one of the following reasons for adjusting inventory quantities:

Reason	Description
<code>correction</code>	Used to correct an inventory error or as a general adjustment reason.
<code>cycle_count_available</code>	Used to specify an adjusted inventory count due to a discrepancy between the actual inventory quantity and previously recorded inventory quantity.
<code>damaged</code>	Used to remove units from inventory count due to damage.
<code>movement_created</code>	Used to specify that an inventory transfer or a purchase order has been created.
<code>movement_updated</code>	Used to specify that an inventory transfer or a purchase order has been updated.
<code>movement_received</code>	Used to specify that an inventory transfer or a purchase order has been received.
<code>movement_canceled</code>	Used to specify that an inventory transfer or a purchase order has been canceled.

Reason	Description
<code>other</code>	Used to specify an alternate reason for the inventory adjustment.
<code>promotion</code>	Used to remove units from inventory count due to a promotion or donation.
<code>quality_control</code>	Used to specify that on-hand units that aren't sellable because they're currently in inspection for quality purposes.
<code>received</code>	Used to specify inventory that the merchant received.
<code>reservation_created</code>	Used to reserve, or temporarily set aside unavailable units.
<code>reservation_deleted</code>	Used to remove the number of unavailable units that have been reserved.
<code>reservation_updated</code>	Used to update the number of unavailable units that have been reserved.
<code>restock</code>	Used to add a returned unit back to available inventory so the unit can be resold.
<code>safety_stock</code>	Used to specify that on-hand units are being set aside to help guard against overselling.
<code>shrinkage</code>	Used when actual inventory levels are less than recorded due to theft or loss.

The following example shows how to set the on-hand inventory quantity to 102. The response shows the delta of incrementing the on-hand inventory quantity by 1 because the previous inventory count in the `on_hand` state was 101.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL mutation



```

1  mutation {
2    inventorySetQuantities(input: {
3      # The reason for adjusting the inventory quantity.
4      reason: "correction",
5      # A freeform URI that represents why the inventory change happened.
6      referenceDocumentUri: "gid://shopify/Order/1974482927638",
7      # The input that's required to set the inventory quantity.
8      quantities: [

```

```

 9      {
10          # The ID of the inventory item.
11          inventoryItemId: "gid://shopify/InventoryItem/32889739542550",
12          # The ID of the location where the inventory is stocked.
13          locationId: "gid://shopify/Location/35239591958",
14          # The quantity of inventory to set.
15          quantity: 102
16          # The last known quantity of the item at the location (used for
17          changeFromQuantity: 101
18      }
19  ]
20  }
21  )
22  # An idempotency key to protect against duplicate adjustments caused
23  @idempotent(key: "4f5b6ebf-143c-4da5-8d0f-fb8553bfd85d") {
24      inventoryAdjustmentGroup {
25          id
26          changes {
27              name
28              delta
29              quantityAfterChange
30          }
31          reason
32          referenceDocumentUri
33      },
34      userErrors {
35          message
36          code
37          field
38      }
39  }
40  }

```

JSON response



```

 1  {
 2      "data": {
 3          "inventorySetQuantities": {
 4              "inventoryAdjustmentGroup": {
 5                  "id": "gid://shopify/InventoryAdjustmentGroup/32766566924310"
 6                  "changes": [
 7                      {
 8                          "name": "available",
 9                          "delta": 1,
10                          "quantityAfterChange": null
11                      }
12                  ],
13                  "reason": "Inventory correction",

```

```
14         "referenceDocumentUri": "gid://shopify/Order/1974482927638"
15     },
16     "userErrors": []
17 }
18 }
```

Adjust inventory quantities

You can use the [inventoryAdjustQuantities](#) mutation to increment or decrement the total amount of inventory that's in the `available`, `damaged`, `quality_control`, `reserved`, or `safety_stock` state at a location. For example, a merchant might need to adjust inventory quantities in the following cases:

State that needs adjustments	Use cases
<code>available</code>	- Inventory was received outside of a purchase order. - Inventory needs to be adjusted in alignment with a completed stocktake.
<code>damaged</code>	- Inventory can't be sold due to damage. - Inventory isn't usable due to physical deterioration.
<code>quality_control</code>	- Inventory isn't meeting a specified quality grade. - Inventory is being measured to meet quality standards.
<code>reserved</code>	- Inventory that is on hold for a customer. - Inventory is being sent to a warehouse, but is not yet available. - Inventory is allocated to an order by a third-party logistics provider that Shopify is unaware of, which means that the inventory isn't in the <code>committed</code> state.
<code>safety_stock</code>	- Inventory has an extra quantity that's being stored in a warehouse. - Inventory is being set aside to insure against fluctuations in demand.

The following example shows how to increase the quantity of available inventory units by 2 at a location. The response shows the delta of incrementing the available

inventory quantity by 2.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL mutation



```

1  mutation {
2    inventoryAdjustQuantities(input: {
3      # The inventory state to receive an adjustment.
4      name: "available",
5      # The reason for adjusting the inventory quantity of the specified
6      reason: "correction",
7      # A freeform URI that represents why the inventory change happened.
8      referenceDocumentUri: "gid://shopify/Order/1974482927638",
9      # The adjustments to the inventory quantity.
10     changes: [{
11       # The ID of the inventory item.
12       inventoryItemId: "gid://shopify/InventoryItem/32889739542550",
13       # The ID of the location where the inventory item is stocked.
14       locationId: "gid://shopify/Location/35239591958",
15       # The inventory adjustment. A positive number (for example, 2) in
16       delta: 2,
17       # The last known quantity of the item at the location (used for c
18       changeFromQuantity: 5
19     }]])
20     # An idempotency key to protect against duplicate adjustments cause
21     @idempotent(key: "4f5b6ebf-143c-4da5-8d0f-fb8553bfd85d") {
22     inventoryAdjustmentGroup {
23       createdAt
24       reason
25       app {
26         id
27       }
28       changes {
29         name
30         delta
31         quantityAfterChange
32       }
33     }
34     userErrors {
35       field
36       message
37     }
38   }
39 }

```

JSON response



```

1  {

```

```

2    "data": {
3      "inventoryAdjustQuantities": {
4        "inventoryAdjustmentGroup": {
5          "createdAt": "2022-11-18T00:59:15Z",
6          "reason": "Inventory correction",
7          "app": {
8            "id": "gid://shopify/App/1830279"
9          },
10         "changes": [
11           {
12             "name": "available",
13             "delta": 2,
14             "quantityAfterChange": null
15           }
16         ]
17       },
18       "userErrors": []
19     }
20   }
21 }

```

Using `referenceDocumentUri` for inventory traceability

The `referenceDocumentUri` field is a freeform URI that indicates why and what caused an inventory change to happen. It's a powerful way to enable complete audit trails for inventory adjustments. `referenceDocumentUri` transforms inventory adjustments from a "black box" into a transparent system where you can trace every change back to its source.

Note: The main difference between a URL and a URI is that a URL specifies the location of a resource on the internet, while a URI can be used to identify any type of resource. All URLs are URIs, but not all URIs are URLs. In this context, you can use either URLs (like `https://myapp.com/order/12345`) or other URI formats (like `gid://myapp/Order/12345`).

`referenceDocumentUri` is accepted on the GraphQL Admin API's [inventoryAdjustQuantities](#), [inventorySetQuantities](#), and [inventoryMoveQuantities](#) mutations.

Why use `referenceDocumentUri`?

Using `referenceDocumentUri` provides the following for merchants:

- Complete audit trail: Trace every inventory change back to the system and document that initiated it.
- Context for changes: Understand not just what changed, but why it changed by linking to the specific business document or transaction.

- **Compliance requirements:** Satisfy regulatory requirements with traceable inventory adjustments.
- **Quick troubleshooting:** Instantly identify which system made each adjustment during discrepancy investigations.

Using `referenceDocumentUri` provides the following for app devs:

- **Brand visibility:** Your app name appears directly in the Shopify admin inventory history
- **Reduced support tickets:** Merchants can self-serve by understanding which system made adjustments
- **Clear accountability:** Connect each inventory change to its originating business logic, making it easier to debug and explain app behavior

Format specification

While the `referenceDocumentUri` field accepts various formats including URLs, our preferred format is the Global ID (GID) format. For example: `gid://warehouse-app/PurchaseOrder/P0-2024-001`

```
1 gid://[namespace]/[entity]/[id]
```



You can add optional query parameters to include additional context like location, user, or timestamp:

```
1 gid://[namespace]/[entity]/[id]?location=west&user=12345
```



Where your URI appears

The `referenceDocumentUri` is visible in two key places:

1. **Inventory adjustment history** - Merchants see your URI in the adjustment history page, providing immediate context for each change
2. **Analytics reports** - The new inventory adjustment reports in Analytics > Reports include your URI for comprehensive tracking

Benefits of using GID format

- **Enhanced popover experience:** When using GID format (e.g., `gid://appName/PurchaseOrder/123456789`), Shopify automatically parses it to display a meaningful title in the adjustment history popover
- **Consistent structure:** Uniform format across all your integrations

- **Clear identification:** Namespace and entity are immediately recognizable
- **Future-ready:** Better support for upcoming features and integrations

Examples of accepted formats

- `gid://my-wms-app/PurchaseOrder/PO-2024-001` (Preferred GID format)
- `gid://3pl-system/CycleCount/CC-2024-0125` (Preferred GID format)
- `gid://pos-app/Transaction/TXN-STORE1-98765?location=west` (Preferred GID format with query parameter)
- `gid://erp/StockAdjustment/ADJ-2024-001` (Preferred GID format)
- `https://myapp.com/order/12345` (URL format - also accepted)
- `id://appChange/12345` (Custom ID format - also accepted)

GID format validation regex (if using GID format)

```
1 const validGidFormat = /^gid:\/\/\/[^\//]+\:\/\/[^\//]+\:\/\/[^\//?]+\((\?.*)?\)$/;
2 const isValidGid = validGidFormat.test(yourUri);
```



Integration examples

Here are example `referenceDocumentUri` formats for common integration types:

WMS (Warehouse Management System) Integration:

- Purchase orders: `gid://warehouse-app/PurchaseOrder/PO-2024-001`
- Stock transfers: `gid://warehouse-app/Transfer/TR-2024-1234`
- Stock adjustments: `gid://warehouse-app/Adjustment/ADJ-2024-567`

3PL (Third-Party Logistics) Integration:

- Cycle counts: `gid://3pl-system/CycleCount/CC-2024-0125`
- Receiving: `gid://3pl-system/Receipt/REC-2024-789`
- Returns processing: `gid://3pl-system/Return/RET-2024-012`

POS (Point of Sale) Integration:

- Sales transactions: `gid://pos-app/Transaction/TXN-STORE1-98765`
- Stock takes: `gid://pos-app/StockTake/ST-STORE1-2024-03`
- Transfers between stores: `gid://pos-app/Transfer/XFER-ST1-ST2-456`

ERP Integration:

- Sync jobs: `gid://erp-connector/SyncJob/SYNC-2024-11-21-001`
- Stock adjustments: `gid://erp/StockAdjustment/ADJ-2024-001`
- Manufacturing orders: `gid://erp/ManufacturingOrder/MO-2024-789`

For complete mutation examples, see the [inventoryAdjustQuantities](#), [inventorySetQuantities](#), and [inventoryMoveQuantities](#) reference documentation.

Best practices

1. Prefer GID format for consistency: While other formats are accepted, using GID format provides consistency and future compatibility
2. Use meaningful namespaces: Choose a namespace that clearly identifies your app or system (e.g., `warehouse-app`, `erp-connector`)
3. Use descriptive entity types: Make it clear what type of document caused the adjustment (e.g., `PurchaseOrder`, `CycleCount`, `StockTransfer`)
4. Include relevant metadata in IDs: Consider including dates, locations, or user information:
 - `gid://inventory-manager/SafetyStock/SS-2024-11-21-loc-456`
 - `gid://wms/Adjustment/ADJ-2024-11-21-user-123`
5. Maintain consistency: Use the same format across all your integrations for easier troubleshooting
6. Document your URIs: Keep internal documentation of your URI patterns for support teams

Migrating existing integrations

To add `referenceDocumentUri` to your existing integrations:

1. Audit current usage: Review your existing inventory adjustment code to identify where `referenceDocumentUri` is missing or incorrectly formatted
2. Plan your namespace: Choose a consistent namespace for your app (avoid generic names like "app" or "system")
3. Update incrementally: You don't need to update all your code at once. Start with new features and high-volume adjustments
4. Common migration patterns to GID format (recommended but not required):

```
1 // Old: No referenceDocumentUri
2 referenceDocumentUri: undefined
3
4 // Using URLs (still supported)
5 referenceDocumentUri: "https://myapp.com/order/12345"
6 // Recommended: Convert to GID format for consistency
7 referenceDocumentUri: "gid://myapp/Order/12345"
8
9 // Old: Using the app ID directly
10 referenceDocumentUri: "gid://shopify/App/123456"
11 // Recommended: Use meaningful references
12 referenceDocumentUri: "gid://myapp/StockAdjustment/ADJ-2024-001"
```

5. Test thoroughly: Verify that your new URIs are accepted by the API and appear correctly in merchant inventory histories

For examples of using `referenceDocumentUri` with inventory mutations, see:

- [inventoryAdjustQuantities examples](#)
- [inventorySetQuantities examples](#)
- [inventoryMoveQuantities examples](#)

Move inventory quantities between states

You can use the `inventoryMoveQuantities` mutation to transition inventory quantities between the `available` state and the `reserved`, `damaged`, `safety_stock`, and `quality_control` states, or between unavailable states (`reserved`, `damaged`, `safety_stock`, and `quality_control`).

For example, a merchant might need to move inventory quantities between states in the following cases:

From state	To state	Description
<code>available</code>	<code>reserved</code>	Inventory is currently being inspected or repaired. Use draft orders instead of moving inventory manually. If a merchant is using Shopify POS and needs to set inventory aside for a customer, then you should use a draft order rather than manually moving inventory to the <code>reserved</code> state.

From state	To state	Description
<code>available</code>	<code>damaged</code>	A third-party logistics provider finds and reports damaged inventory.
<code>available</code>	<code>safety_stock</code>	Inventory is being set aside to insure against fluctuations in demand.
<code>available</code>	<code>quality_control</code>	Inventory is being measured to meet quality standards.
<code>reserved</code>	<code>available</code>	A stocktake was completed and inventory needs to be adjusted accordingly.
<code>damaged</code>	<code>available</code>	A previously damaged inventory unit that was set aside has been repaired.
<code>safety_stock</code>	<code>available</code>	Extra inventory is available to help with fluctuations in demand.
<code>quality_control</code>	<code>available</code>	An inventory unit has met quality standards and is ready to be sold.

Note

Inventory quantities in a `reserved`, `damaged`, `safety_control`, or `quality_control` state display as **Unavailable** to merchants that are tracking inventory in the Shopify admin.

The following example shows how to move a quantity of 2 inventory units from the `available` to the `reserved` state.

POST https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json

GraphQL mutation



```

1  mutation {
2    inventoryMoveQuantities(input: {
3      # The reason for moving a certain quantity of inventory units to
4      reason: "correction",
5      # A freeform URI that represents why the inventory change happened
6      referenceDocumentUri: "uri://example.com/some/external/reference",
7      # The movement of quantities between inventory states
8      changes: [{
9        inventoryItemId: "gid://shopify/InventoryItem/32889739542550",
10       from: {
11         # The state that you want to move the inventory quantities fr

```

```

12     name: "available",
13     # The location where the inventory is stocked.
14     locationId: "gid://shopify/Location/35239591958",
15     # The last known available quantity of the item at the locati
16     changeFromQuantity: 5
17   }
18   to: {
19     # The state that you want to move the inventory quantities to
20     name: "reserved",
21     # The ID of the location where the inventory is stocked.
22     locationId: "gid://shopify/Location/35239591958",
23     # A freeform URI that represents what changed the inventory c
24     ledgerDocumentUri: "uri://example.com/some/external/reference
25     # The last known reserved quantity of the item at the locatio
26     changeFromQuantity: 10
27   }
28   # The quantity of inventory units that you want to move to a diff
29   quantity: 2
30 }]])
31 # An idempotency key to protect against duplicate adjustments call
32 @idempotent(key: "4f5b6ebf-143c-4da5-8d0f-fb8553bfd85d") {
33   inventoryAdjustmentGroup {
34     createdAt
35     reason
36     app {
37       id
38     }
39     changes {
40       name
41       delta
42       quantityAfterChange
43     }
44   }
45   userErrors {
46     field
47     message
48   }
49 }
50 }

```

JSON response

```

1  {
2    "data": {
3      "inventoryMoveQuantities": {
4        "inventoryAdjustmentGroup": {
5          "createdAt": "2022-11-18T21:06:46Z",
6          "reason": "Inventory correction",
7          "app": {

```



```

8      "id": "gid://shopify/App/1830279"
9    },
10    "changes": [
11      {
12        "name": "available",
13        // The available inventory quantity is decremented by 2.
14        "delta": -2,
15        // This field always returns `null` immediately after run
16        "quantityAfterChange": null
17      },
18      {
19        "name": "reserved",
20        // The reserved inventory quantity is incremented by 2.
21        "delta": 2,
22        // This field always returns `null` immediately after run
23        "quantityAfterChange": null
24      }
25    ]
26  },
27  "userErrors": []
28 }
29 }
30 }
```

Schedule changes to inventory quantities

You can schedule changes to inventory quantities using the [inventorySetScheduledChanges](#) mutation. This involves defining the dates associated with upcoming changes to inventory quantities.

For example, if an app user creates a purchase order and adds incoming quantities for a specified inventory item at a location, then they can also create a scheduled change that states the date that the quantities are expected to be transitioning from `incoming` to `available`. Information about scheduled changes to inventory quantities can help merchants make better buying or selling decisions.

⚠ Caution

Scheduled changes are being phased out: [inventorySetScheduledChanges](#) isn't available in the 2026-07 Admin API, and [scheduledChanges](#) on [InventoryLevel](#) is deprecated. The mutation never changed inventory quantities on its own either. To change quantities, use [inventoryAdjustQuantities](#), [inventorySetQuantities](#), or [inventoryMoveQuantities](#) when the change happens.

The following example shows how to create a scheduled change that defines the date that quantities are expected to be transitioning from `incoming` to `available`.

POST `https://{shop}.myshopify.com/admin/api/{api_version}/graphql.json`

GraphQL mutation



```

1  mutation inventorySetScheduledChanges {
2    inventorySetScheduledChanges(input: {
3      # The reason for moving a certain quantity of inventory units to
4      reason: "correction",
5      items: [
6        {
7          # The ID of the inventory item.
8          inventoryItemId: "gid://shopify/InventoryItem/32889739542550"
9          # The ID of the location where the inventory is stocked.
10         locationId: "gid://shopify/Location/35239591958",
11         # A freeform URI that represents what changed the inventory c
12         ledgerDocumentUri: "uri://example.com/some/external/reference"
13         # An array that defines the date that the quantities are expected
14         scheduledChanges: [
15           {
16             expectedAt: "2023-12-11T23:37:38Z"
17             fromName: "incoming",
18             toName: "available"
19           }
20         ]
21       }
22     ],
23     # A freeform URI that represents why the inventory change happened
24     referenceDocumentUri: "uri://example.com/some/external/reference"
25   })
26   # An idempotency key to protect against duplicate adjustments calculated
27   @idempotent(key: "4f5b6ebf-143c-4da5-8d0f-fb8553bfd85d") {
28     scheduledChanges {
29       expectedAt
30       fromName
31       toName
32       quantity
33       ledgerDocumentUri
34       inventoryLevel {
35         id
36         item {
37           variant {
38             id
39             displayName
40           }
41         }
42         location {

```

```

43         name
44         id
45     }
46 }
47 }
48     userErrors {
49         message
50         field
51     }
52 }
53 }

```

JSON response



```

1  {
2    "data": {
3      "inventorySetScheduledChanges": {
4        "scheduledChanges": [
5          {
6            "expectedAt": "2023-12-11T23:37:38Z",
7            "fromName": "incoming",
8            "toName": "available",
9            "quantity": 4,
10           "ledgerDocumentUri": "uri://example.com/some/external/refer
11           "inventoryLevel": {
12             "id": "gid://shopify/InventoryLevel/69407473686?inventory
13             "item": {
14               "variant": {
15                 "id": "gid://shopify/ProductVariant/31365818744854",
16                 "displayName": "French Bulldog on a swing"
17               }
18             },
19             "location": {
20               "name": "180 Switchmen Street",
21               "id": "gid://shopify/Location/35239591958"
22             }
23           }
24         ]
25       },
26       "userErrors": []
27     }
28   },
29   "extensions": {
30     "cost": {
31       "requestedQueryCost": 14,
32       "actualQueryCost": 14,
33       "throttleStatus": {
34         "maximumAvailable": 1000.0,
35         "currentlyAvailable": 986,

```

```
36         "restoreRate": 50.0
37     }
38 }
39 }
40 }
```

Compare and swap

Some inventory mutations, such as [inventoryAdjustQuantities](#), [inventoryMoveQuantities](#), [inventorySetQuantities](#), and [productVariantsBulkUpdate](#), implement compare and swap.

The `changeFromQuantity` field enables concurrency control by allowing you to specify the quantity that you expect to be present before your change is applied. If the current quantity doesn't match the value that you provide, then the mutation fails with a `CHANGE_FROM_QUANTITY_STALE` error, preventing unintended overwrites.

Example of why compare and swap is necessary

The following example uses `inventorySetQuantities` to demonstrate why compare and swap is necessary:

1. Suppose that you have 100 red shoes available in one of your store locations.
2. You query the current available inventory from Shopify's APIs and determine that there are 100 units.
3. You want to adjust inventory by -10 using `inventorySetQuantities`.
4. Before you can do that, another process changes the inventory. 20 units are sold, so there are now 80 units.
5. You want to adjust inventory by -10 (thinking the current quantity is 100), so you set the inventory to 90 using `inventorySetQuantities`.
6. Result: The final available inventory value is 90, but it should be 70 (20 units lost).

To prevent this scenario, you should pass in `changeFromQuantity: 100`. When you call `inventorySetQuantities`, the current available quantity (80) is compared against the `changeFromQuantity` (100). Because they don't match, the process has changed the current quantity concurrently, so the request is denied.

Opting out of compare and swap

We recommend that you implement compare and swap to avoid inventory inaccuracies. However, there is one scenario where you should opt out:

- If you don't care about the current quantity because you're the source of truth, then you don't need to implement compare and swap and should opt out. For example, you might know definitively what the new quantity should be, or you might need to ensure your adjustment is applied regardless of any concurrent changes made by other processes.

Next steps

- Learn about other [types of fulfillment apps](#) that you can create to integrate into order management, fulfillment, and returns workflows.

Was this page helpful? [Yes](#) [No](#)

Updates

Developer changelog
Shopify Editions

Legal

Terms of service
API terms of use
Privacy policy
Partners Program Agreement

Business growth

Shopify Partners Program
Shopify App Store
Shopify Academy

Shopify

About Shopify
Shopify Plus
Careers
Investors

Press and media